

Backups de PostgreSQL — H.A.M.M.E.R. POS

Guía completa para crear, restaurar y automatizar backups de la base de datos PostgreSQL en producción.

Tabla de contenidos

1. [Requisitos previos](#)
 2. [Estructura de archivos](#)
 3. [Crear un backup manual](#)
 4. [Restaurar un backup](#)
 5. [Automatización con cron](#)
 6. [Sistema de retención](#)
 7. [Backup remoto \(S3/rclone\)](#)
 8. [Mejores prácticas](#)
 9. [Troubleshooting](#)
-

Requisitos previos

- Docker y Docker Compose instalados
- Servicios de producción corriendo (`docker compose -f docker-compose.production.yml up -d`)
- Archivo `.env.production` configurado con las credenciales de PostgreSQL
- Permisos de ejecución en los scripts (`chmod +x scripts/*.sh`)

Estructura de archivos

```
hammer-pos/
├── scripts/
│   ├── backup-postgres.sh    # Script de backup
│   └── restore-postgres.sh   # Script de restauración
├── backups/                  # Directorio de backups (montado en contenedor)
│   ├── hammer_pos_2025-05-12_10-00-00.sql.gz
│   ├── hammer_pos_2025-05-12_22-00-00.sql.gz
│   └── backup.log            # Log de operaciones
├── docs/
│   └── BACKUPS.md            # Esta documentación
└── docker-compose.production.yml # Incluye volumen ./backups:/backups
```

Crear un backup manual

Comando básico

```
cd /ruta/a/hammer-pos
./scripts/backup-postgres.sh
```

Opciones disponibles

Opción	Descripción	Default
<code>--retain N</code>	Número de backups a conservar	30
<code>--dir PATH</code>	Directorio donde guardar los backups	<code>./backups</code>
<code>--file</code>	Archivo docker-compose a usar	<code>docker-compose.production.yml</code>
<code>--env</code>	Archivo de variables de entorno	<code>.env.production</code>
<code>-h/--help</code>	Mostrar ayuda	—

Ejemplos

```
# Backup con retención de 7 días
./scripts/backup-postgres.sh --retain 7

# Guardar en directorio personalizado
./scripts/backup-postgres.sh --dir /mnt/external/backups

# Usar archivo compose diferente (ej: staging)
./scripts/backup-postgres.sh --file docker-compose.staging.yml --env .env.staging
```

¿Qué genera?

- Archivo: `backups/hammer_pos_YYYY-MM-DD_HH-MM-SS.sql.gz`
- Log: `backups/backup.log`
- El backup usa `pg_dump` con las opciones `--clean --if-exists --no-owner --no-privileges` para máxima portabilidad.

Restaurar un backup



Advertencia importante

Restaurar sobre la base de datos de producción DESTRUIRÁ todos los datos actuales. Siempre haga un backup antes de restaurar y considere usar `--temp` para verificar primero.

Listar backups disponibles

```
./scripts/restore-postgres.sh --list
```

Ejemplo de salida:

📁 Backups disponibles en ./backups:

ARCHIVO	TAMAÑO	FECHA
-----	-----	-----
hammer_pos_2025-05-12_22-00-00.sql.gz	2.5MiB	2025-05-12 22:00:00
hammer_pos_2025-05-12_10-00-00.sql.gz	2.4MiB	2025-05-12 10:00:00

Total: 2 backup(s)

Restaurar en base de datos temporal (recomendado)

```
./scripts/restore-postgres.sh --temp backups/hammer_pos_2025-05-12_22-00-00.sql.gz
```

Esto:

1. Crea una DB temporal `hammer_pos_restore_temp`
2. Restaura el backup ahí
3. Permite inspeccionar los datos antes de aplicar en producción

Conectarse a la DB temporal:

```
docker compose -f docker-compose.production.yml exec db \
psql -U hammer -d hammer_pos_restore_temp
```

Eliminar la DB temporal:

```
docker compose -f docker-compose.production.yml exec db \
psql -U hammer -d hammer_pos -c 'DROP DATABASE hammer_pos_restore_temp;'
```

Restaurar en producción

```
./scripts/restore-postgres.sh backups/hammer_pos_2025-05-12_22-00-00.sql.gz
```

Requiere escribir **RESTAURAR PRODUCCION** como confirmación de seguridad.

Opciones del script de restauración

Opción	Descripción
<code>--temp</code>	Restaurar en DB temporal para verificación
<code>--list</code>	Listar backups disponibles
<code>--force</code>	Saltar confirmación (⚠ PELIGROSO)
<code>--file</code>	Archivo docker-compose
<code>--env</code>	Archivo .env

Automatización con cron

Backup diario a las 2:00 AM

```
# Editar crontab del usuario
crontab -e

# Agregar la siguiente línea:
0 2 * * * cd /ruta/a/hammer-pos && ./scripts/backup-postgres.sh >> /var/log/hammer-backup.log 2>&1
```

Backup cada 12 horas

```
0 */12 * * * cd /ruta/a/hammer-pos && ./scripts/backup-postgres.sh --retain 60
```

Backup diario con retención de 7 días

```
0 3 * * * cd /ruta/a/hammer-pos && ./scripts/backup-postgres.sh --retain 7
```

Verificar que cron está funcionando

```
# Ver crontab actual
crontab -l

# Verificar logs de cron
grep CRON /var/log/syslog | tail -20

# Verificar log de backups
tail -20 /ruta/a/hammer-pos/backups/backup.log
```

Sistema de retención

El script de backup incluye un sistema automático de retención:

- **Por defecto:** conserva los últimos **30 backups**
- **Configurable:** con `--retain N`
- **Funcionamiento:** después de cada backup, los archivos más antiguos se eliminan automáticamente
- **Criterio:** se basa en la fecha de modificación del archivo

Estrategias recomendadas

Escenario	Frecuencia	Retención	Comando
Producción estándar	Diario	30 días	<code>--retain 30</code>
Alto volumen de transacciones	Cada 6 horas	28 backups	<code>--retain 28</code>
Espacio limitado	Diario	7 días	<code>--retain 7</code>
Pre-deploy	Manual	Sin límite	(no usar retención automática)

Backup remoto (S3/rclone)

El script de backup incluye secciones comentadas para sincronización remota. Para activar:

Opción A: rclone

```
# Instalar rclone
curl https://rclone.org/install.sh | sudo bash

# Configurar remote (interactivo)
rclone config

# Descomentar las líneas de rclone en scripts/backup-postgres.sh
# y ajustar RCLONE_REMOTE
```

Opción B: AWS S3

```
# Instalar AWS CLI
apt install awscli

# Configurar credenciales
aws configure

# Descomentar las líneas de S3 en scripts/backup-postgres.sh
# y ajustar S3_BUCKET
```

Mejores prácticas

Seguridad

- ☒ Los backups se guardan **fuera del contenedor** en `./backups` (volumen montado)
- ☒ Nunca subir backups al repositorio (agregar `backups/` a `.gitignore`)
- ☒ Cifrar backups antes de enviar a la nube: `gpg -c backup.sql.gz`
- ☒ Restringir permisos del directorio de backups: `chmod 700 ./backups`

Verificación

- ☒ Probar la restauración periódicamente en una DB temporal (`--temp`)
- ☒ Verificar el tamaño del backup (un backup vacío o muy pequeño indica problemas)
- ☒ Revisar `backups/backup.log` regularmente

Almacenamiento

- ☒ Monitorear espacio en disco: `df -h`
- ☒ Los backups comprimidos con gzip (-9) suelen ser 5-10x más pequeños que el SQL plano
- ☒ Usar retención automática para evitar llenar el disco

Antes de actualizaciones

- ☒ **Siempre** hacer un backup manual antes de:
- Actualizar la aplicación
- Ejecutar migraciones
- Cambios de infraestructura

```
# Backup pre-deploy
./scripts/backup-postgres.sh
# ... realizar actualización ...
# Si algo falla:
./scripts/restore-postgres.sh --temp backups/hammer_pos_<timestamp>.sql.gz
```

Troubleshooting

El script dice “contenedor no está corriendo”

```
# Verificar estado de los servicios
docker compose -f docker-compose.production.yml ps

# Iniciar servicios si están detenidos
docker compose -f docker-compose.production.yml --env-file .env.production up -d
```

El backup está vacío (0 bytes)

1. Verificar que PostgreSQL acepta conexiones:

```
bash
docker compose -f docker-compose.production.yml exec db \
pg_isready -U hammer -d hammer_pos
```

2. Verificar credenciales en `.env.production`:

```
bash
grep POSTGRES .env.production
```

3. Intentar `pg_dump` manualmente:

```
bash
docker compose -f docker-compose.production.yml exec db \
pg_dump -U hammer -d hammer_pos | head -20
```

Error “permission denied” al crear backup

```
# Asegurar que el directorio existe y tiene permisos correctos
mkdir -p backups
chmod 755 backups

# Verificar permisos de los scripts
chmod +x scripts/backup-postgres.sh scripts/restore-postgres.sh
```

La restauración falla con errores de SQL

Algunos warnings son normales durante la restauración (ej: “role does not exist”, “schema already exists”). El script usa `--set ON_ERROR_STOP=off` para continuar a pesar de warnings menores.

Si hay errores graves:

1. Restaure primero en DB temporal: `--temp`
2. Revise la salida del script
3. Verifique que el backup no esté corrupto: `gunzip -t backup.sql.gz`

No hay espacio en disco

```
# Ver espacio disponible
df -h

# Ver tamaño de backups
du -sh backups/

# Limpiar manualmente (mantener solo los últimos 5)
ls -t backups/hammer_pos_*.sql.gz | tail -n +6 | xargs rm -f

# Limpiar imágenes Docker no usadas
docker system prune -f
```

Cron no ejecuta el backup

1. Verificar PATH en crontab (agregar al inicio del crontab):

```
PATH=/usr/local/bin:/usr/bin:/bin
```

2. Asegurar rutas absolutas en la línea cron:

```
0 2 * * * cd /home/deploy/hammer-pos && /home/deploy/hammer-pos/scripts/backup-postgres.sh
```

3. Verificar que Docker es accesible desde cron:

```
bash
# El usuario de cron necesita estar en el grupo docker
sudo usermod -aG docker $USER
```

Referencias rápidas

Crear backup

```
./scripts/backup-postgres.sh
```

Listar backups

```
./scripts/restore-postgres.sh --list
```

Restaurar (seguro)

```
./scripts/restore-postgres.sh --temp backups/<archivo>.sql.gz
```

Restaurar (producción)

```
./scripts/backup-postgres.sh # ¡Backup primero!  
./scripts/restore-postgres.sh backups/<archivo>.sql.gz
```